

TGraph と TF1

ーグラフの作成とフィッティングー

20260728

YMAP ROOT講習会

櫻井駿介

自己紹介

櫻井駿介

- 博士：東大宇宙線研（高エネルギーガンマ線）
- 特任助教：大阪公立大学/大阪電気通信大学（超高エネルギー宇宙線）

趣味

- ビリヤード
- 歩くこと（最近は暑すぎる）
- 100 km ウォーキングにまた出たい

今日の目標

基礎目標

- TGraph が作れるようになる
- TF1 でフィットができるようになる

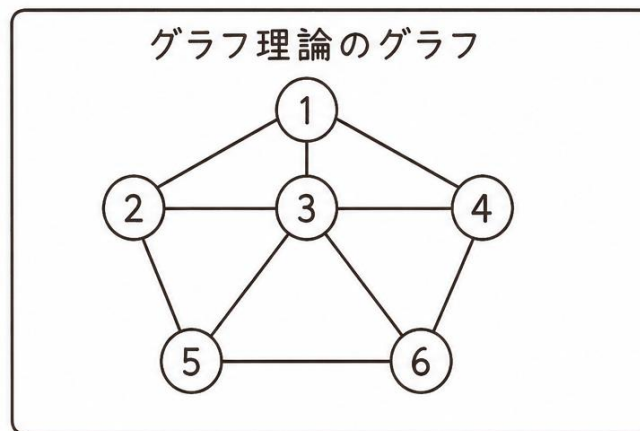
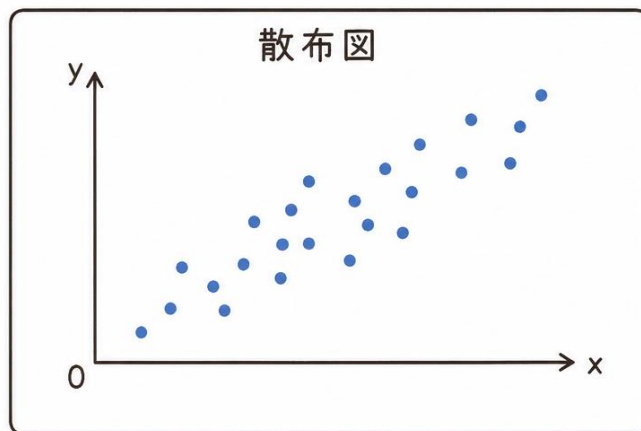
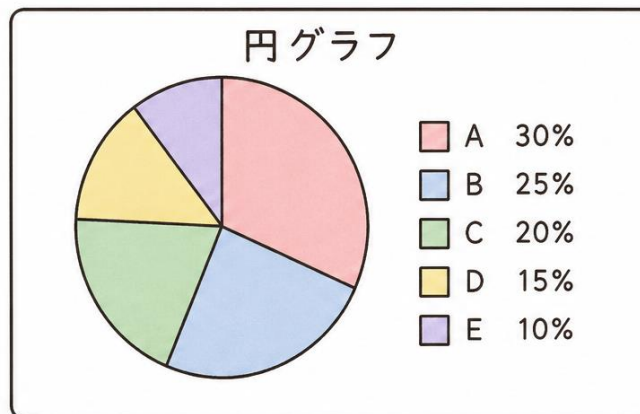
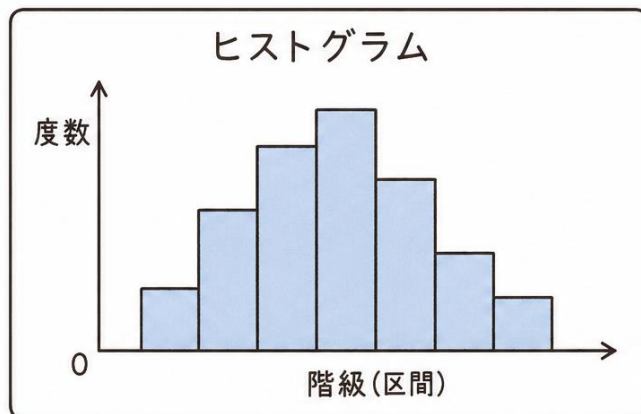
発展目標

- モデルの選び方を学ぶ
- 不確かさについて学ぶ

パート 1：グラフとフィッティングの実例

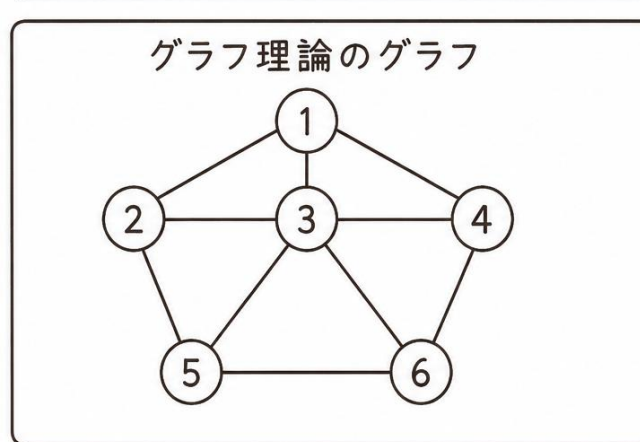
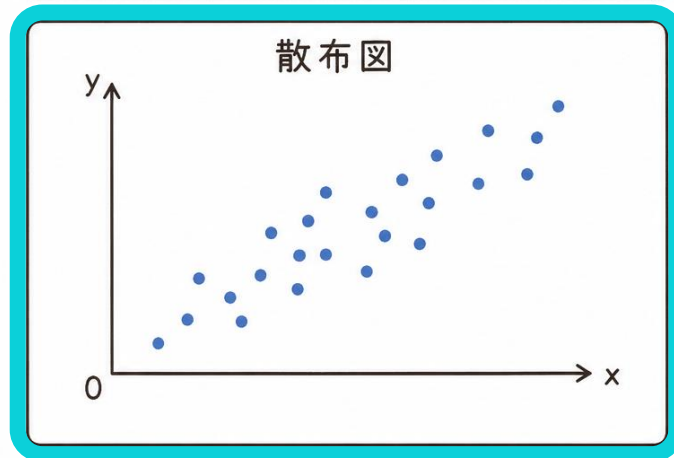
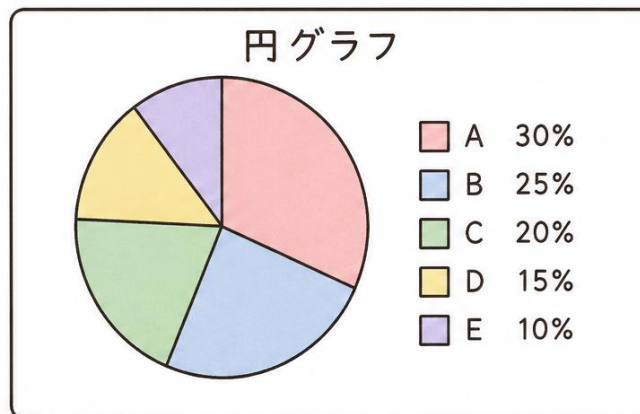
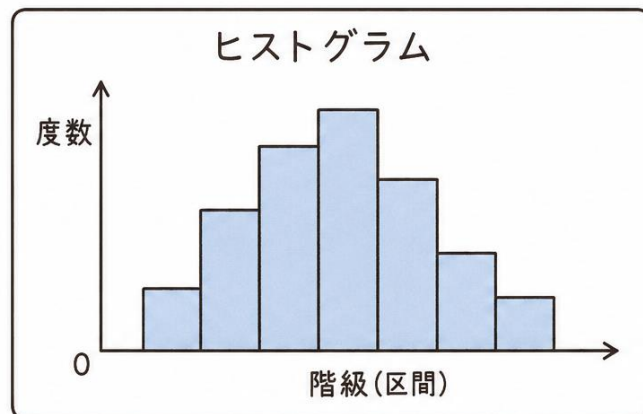
グラフとは

グラフという言葉はあいまい



グラフとは

人によって使い方が違う



今日はこれが**グラフ**

グラフ=データの組

物理屋さんのやりたいこと

モデル化

変数 x と y の間の関係を知りたい

扱いやすい形になってくれると良い

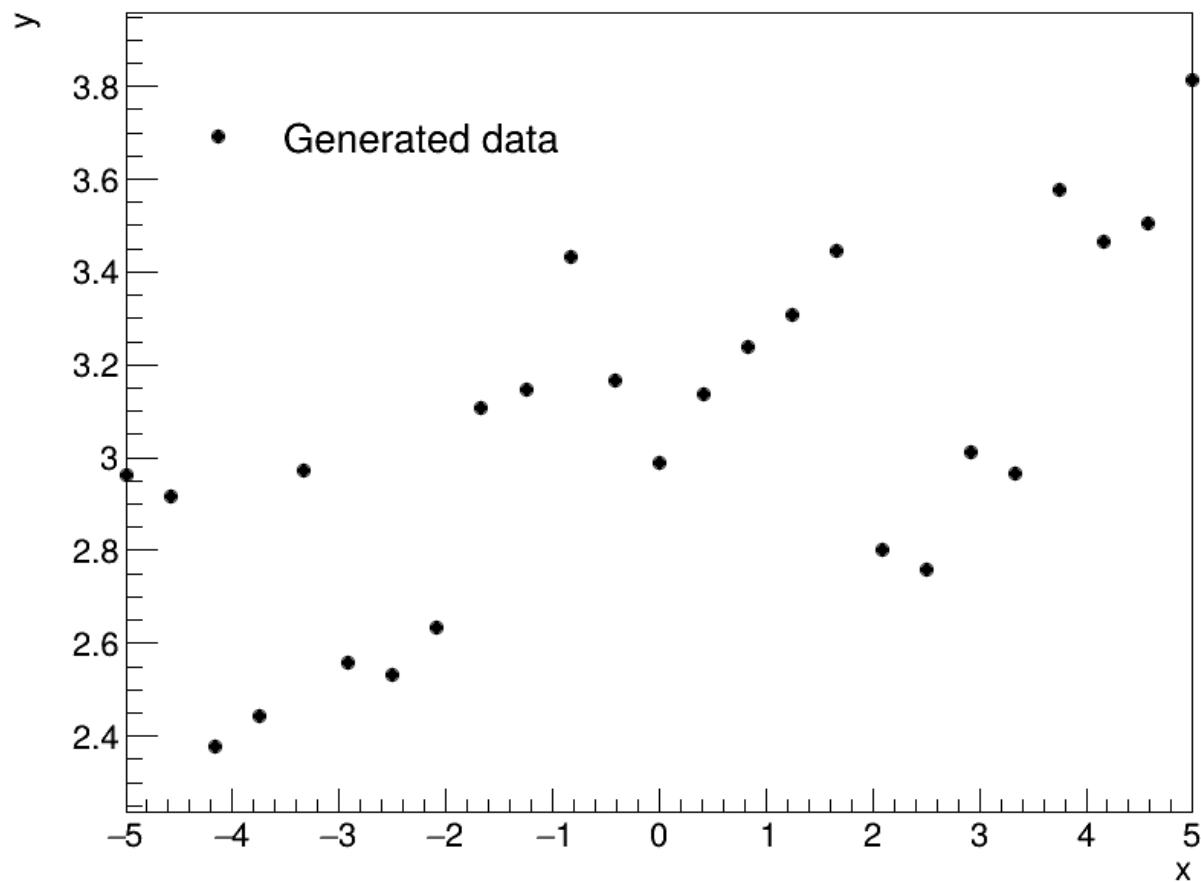
→ 数学的な関数でモデリングする

フィッティング

もっとも良い（妥当な）モデルを決める方法

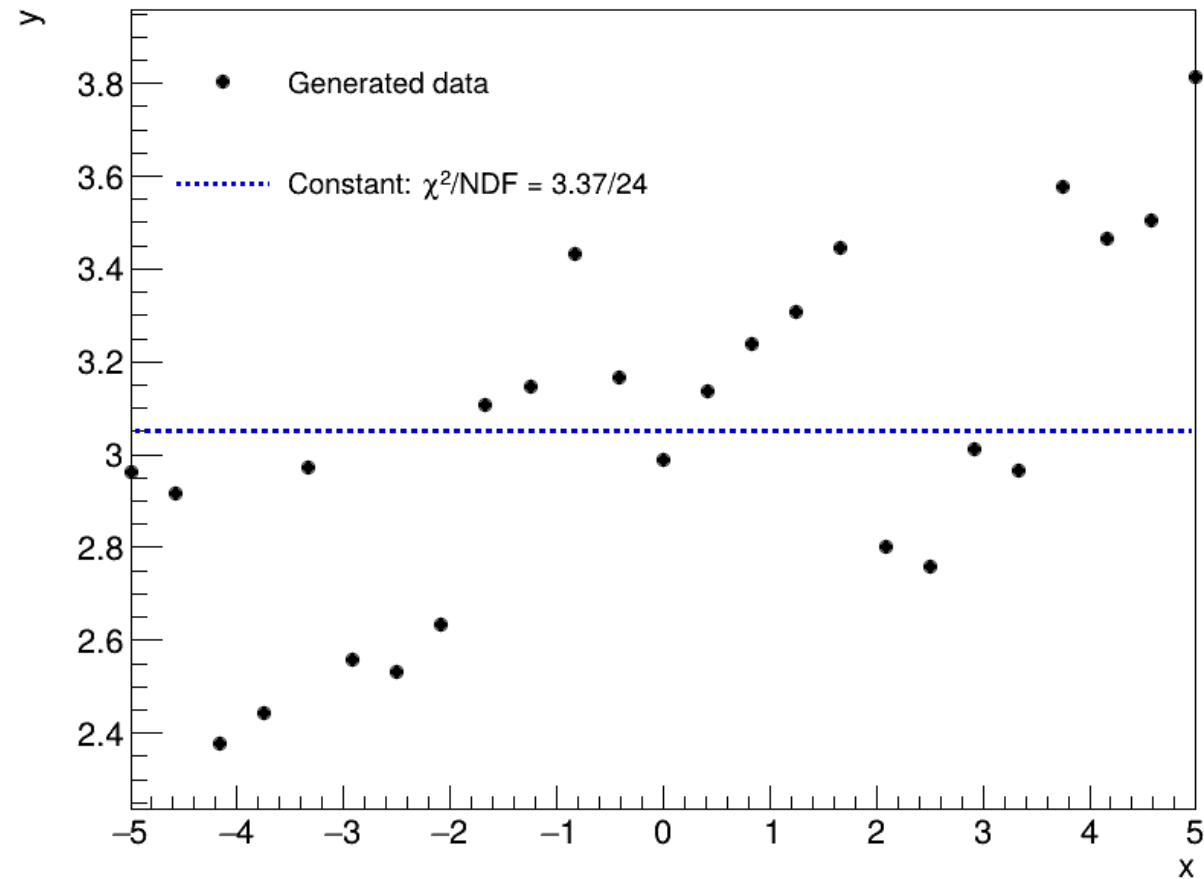
フィッティングの例

Fits with different polynomial orders



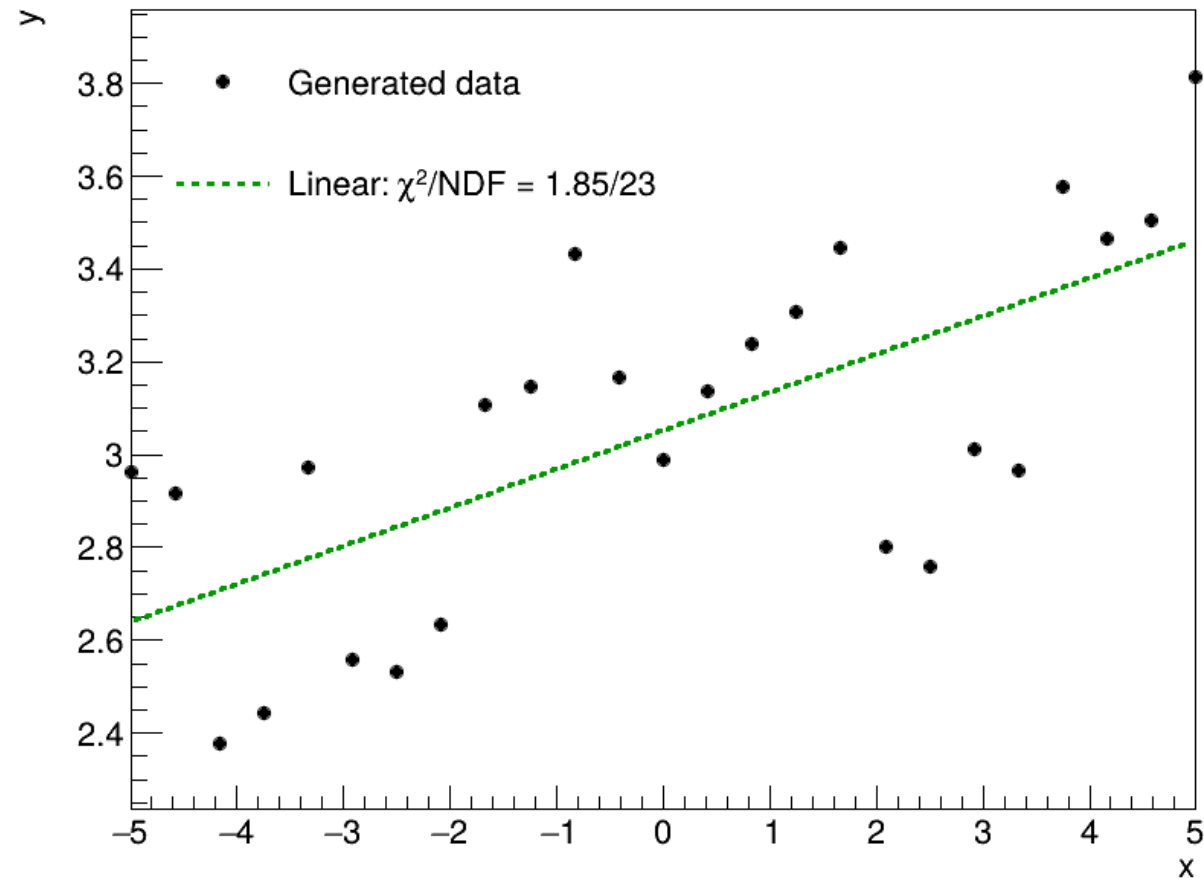
フィッティングの例

Fits with different polynomial orders



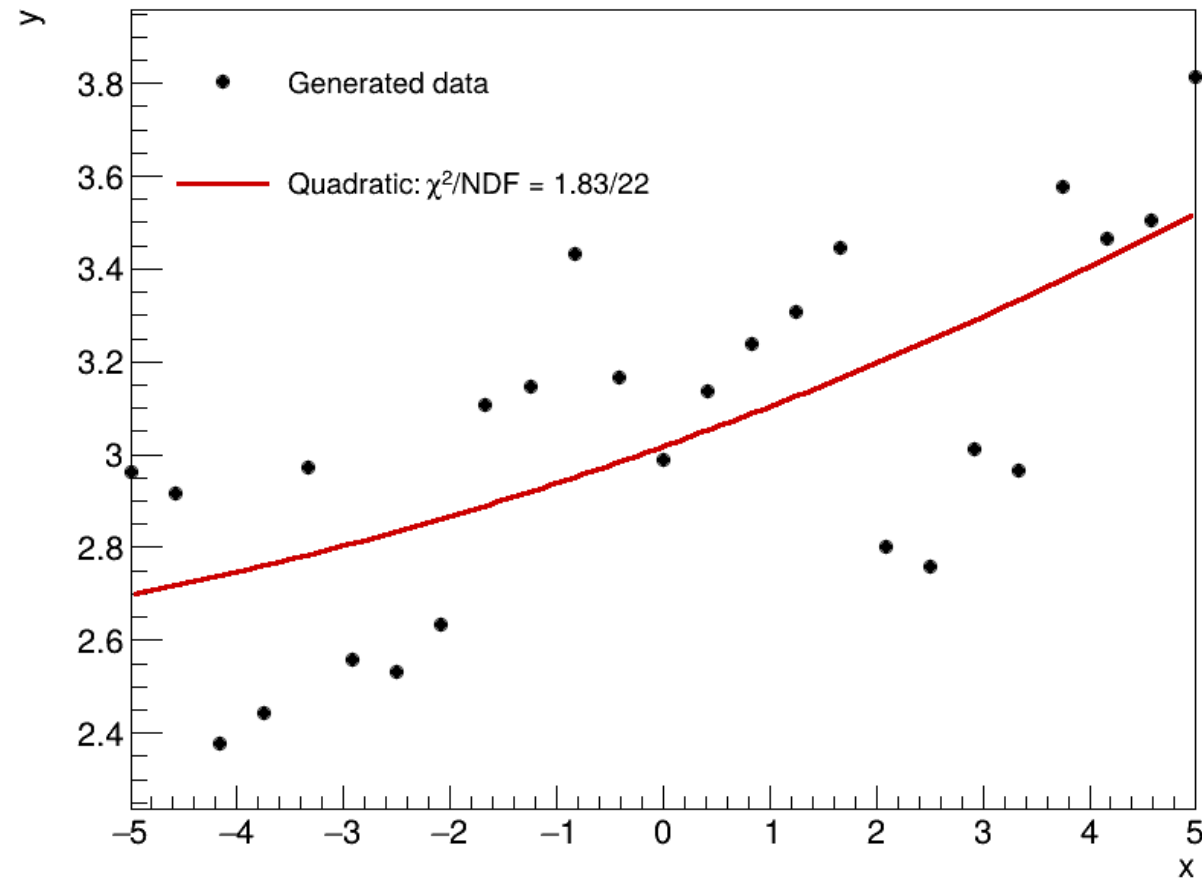
フィッティングの例

Fits with different polynomial orders



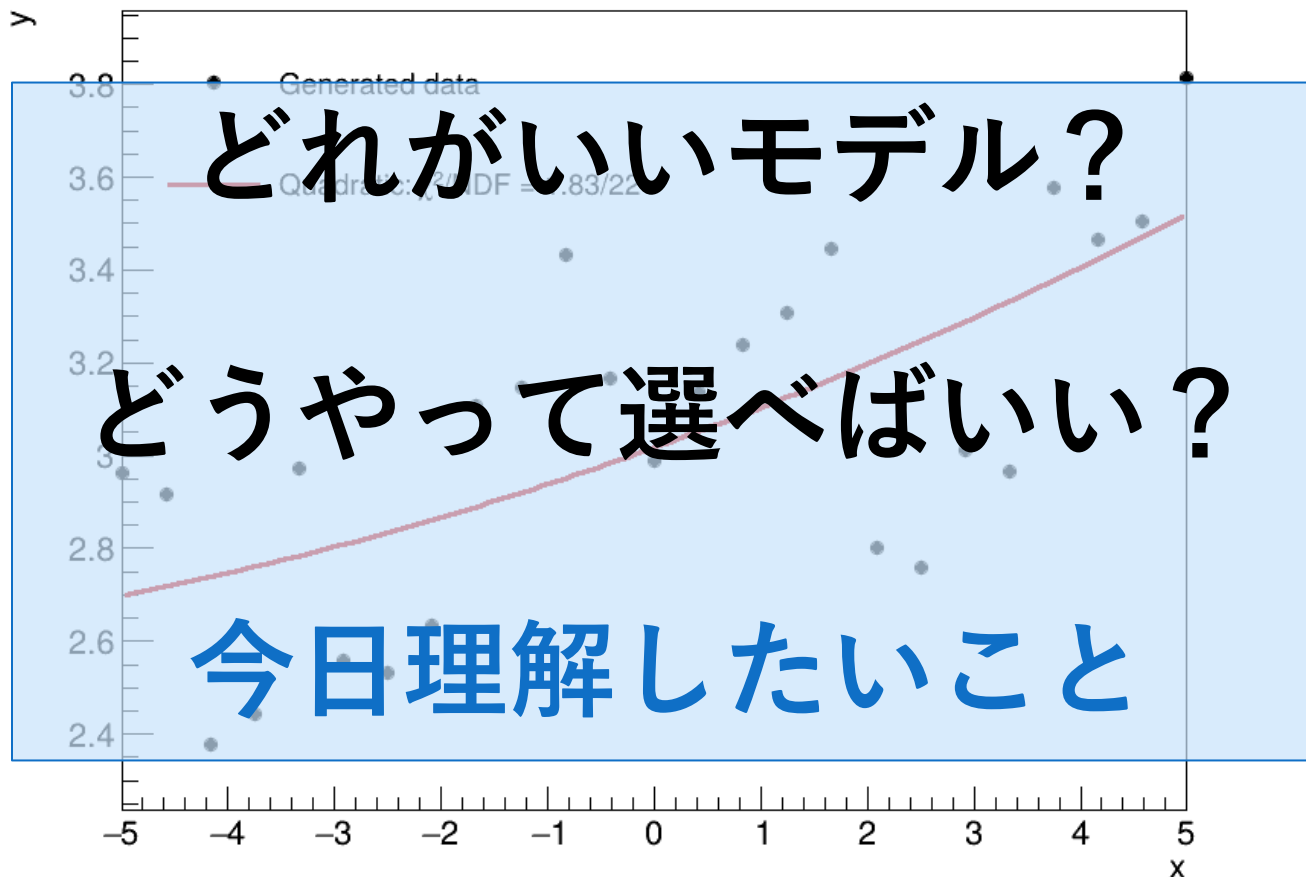
フィッティングの例

Fits with different polynomial orders



フィッティングの例

Fits with different polynomial orders



モデルの選択

モデルの選び方には様々な方法がある。櫻井の理解は大きく2つ

ボトムアップな手法（事前知識が無い時）

- 単純なモデルから始めて、段階的に複雑にする
- なるべく単純なモデルでデータの間係を説明できればOK

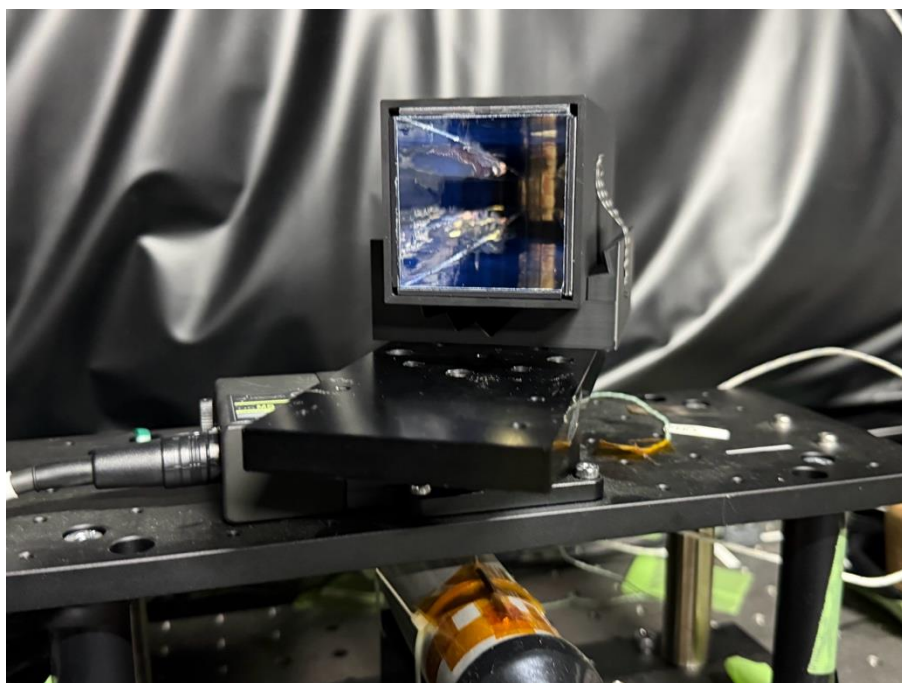
トップダウンな手法（事前知識が有る時）

- ある物理現象を記述するのに妥当な理論的背景（仮定）もしくは経験則がある時、それらをモデルに用いる

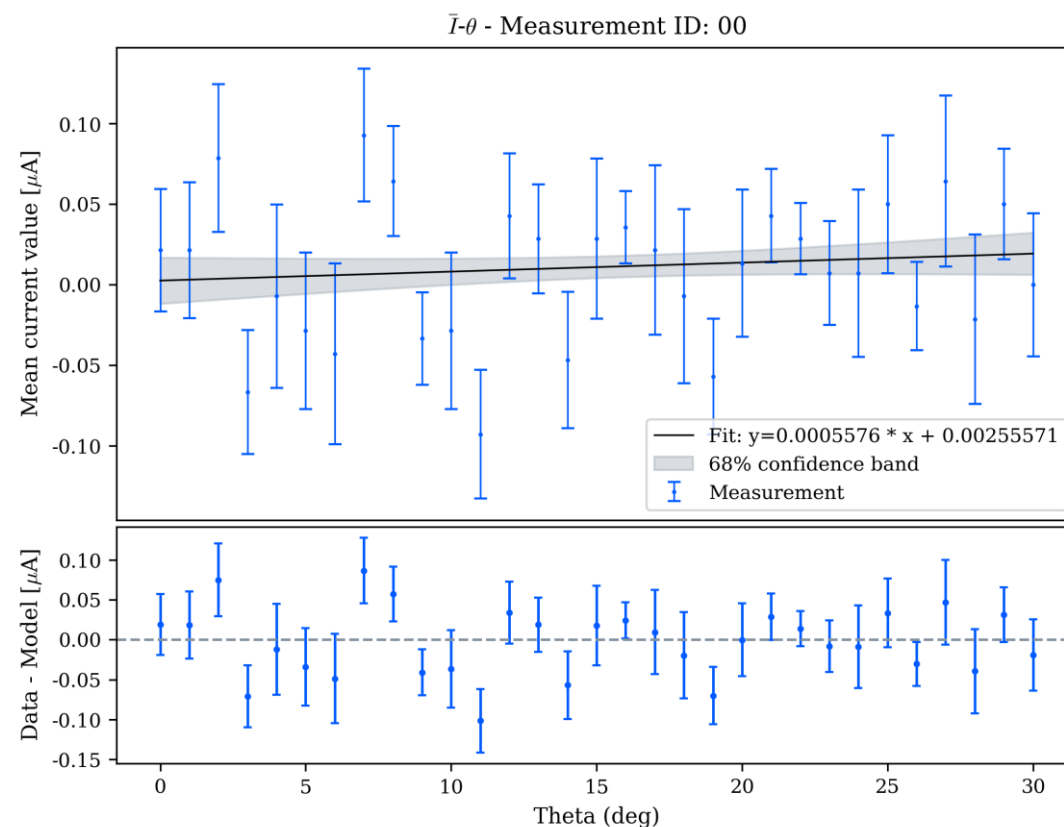
ボトムアップな手法の例

光集光器の角度依存性の評価

- 光を照射していない場合に、背景成分に偏りが無いか知りたい



1次関数である必要は無さそう



ボトムアップな手法の注意点

モデルをむやみに複雑にする必要は無い

- モデルの次元数を増やすほどデータには合うようになる
関数の表現能力は上昇する (e.g. 深層学習)
- フィッティングでは評価指標に組み込まれている (chi2 : 後述)

物理で用いられる考え

- オッカムの剃刀 : 最低限のものだけでものごとを説明する

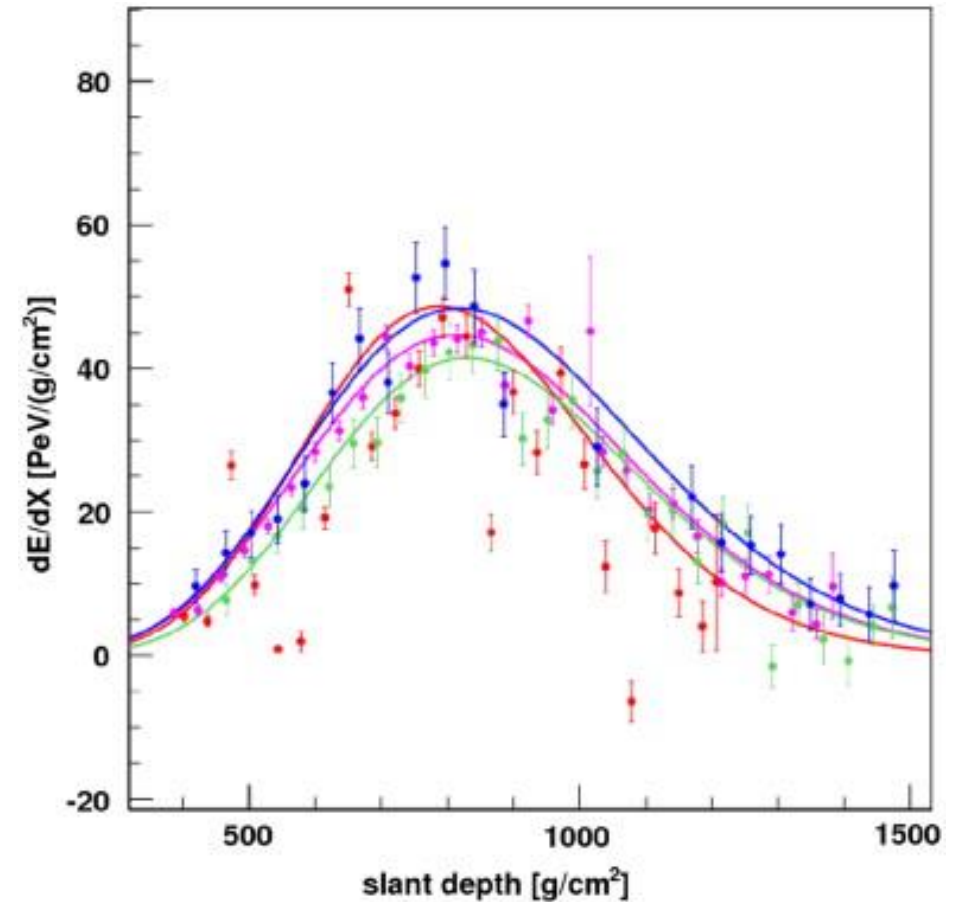
トップダウンな手法の例

宇宙線実験での例

- 空気シャワー（宇宙線による二次粒子）の発達の仕方をモデル化
- Gaisser-Hillas function

$$N(X) = N_{\max} \left(\frac{X - X_0}{X_{\max} - X_0} \right)^{\frac{X_{\max} - X_0}{\lambda}} \exp \left(-\frac{X_{\max} - X}{\lambda} \right)$$

どのような経緯でモデルが使われているか
確かめておく必要がある



<https://doi.org/10.1016/j.nima.2010.10.128>

パート 2 : Hands on with ROOT

TGraph を書いてみる

まずは画面に点を打つところから

```
#include <TCanvas.h>
#include <TGraph.h>
```

C++

```
void graph(){
    TCanvas* c = new TCanvas("c", "Tutorial", 600, 400);
    // プロットしたいデータ
    double x[3] = {0, 1, 2};
    double y[3] = {0, 1, 4};
    // データの大きさとデータを渡す
    TGraph* g = new TGraph(3, x, y);
    // 描画
    g->Draw("a*");
    // 保存
    c->SaveAs("test_TGraph_ROOT.pdf");
}
```

```
import ROOT
import numpy as np
```

Python

```
canvas = ROOT.TCanvas("c", "Tutorial", 600, 400)
# プロットしたいデータ
# 型が指定されていないとエラーが出る場合がある
x = np.array([0, 1, 2], dtype=float)
y = np.array([0, 1, 4], dtype=float)
# データの大きさとデータを渡す
graph = ROOT.TGraph(len(x), x, y)
# 描画
graph.Draw("a*")
# 保存
canvas.SaveAs("test_TGraph_pyroot.pdf")
```

はじめに紹介したデータをグラフにしてみる

```
// -----
// データ生成条件
// -----
const int nPoints = 25;
const double xMin = -5.0;
const double xMax = 5.0;

// 真の二次関数:
//  $y = \text{trueP0} + \text{trueP1} * x + \text{trueP2} * x^2$ 
const double trueP0 = 3.0;
const double trueP1 = 0.25;
const double trueP2 = 0.025;

// データ点に加えるランダムなばらつき。
// TGraphには誤差として設定しない。
const double noiseSigma = 0.50;

// シードを固定して、毎回同じデータを生成する。
TRandom3 random(12345);
std::vector<double> x(nPoints);
std::vector<double> y(nPoints);
for (int i = 0; i < nPoints; ++i) {
    x[i] = xMin + (xMax - xMin) * i / (nPoints - 1.0);
    const double yTrue =
        trueP0
        + trueP1 * x[i]
        + trueP2 * x[i] * x[i];
    y[i] = yTrue + random.Gaus(0.0, noiseSigma);
}
```

C++

```
# -----
# データ生成条件
# -----
n_points = 25
x_min = -5.0
x_max = 5.0

# 真の関数:
#  $y = \text{true\_p0} + \text{true\_p1} * x + \text{true\_p2} * x^2$ 
true_p0 = 3.0
true_p1 = 0.1
true_p2 = 0.01

# データに加えるランダムなばらつき。
# TGraphには誤差として設定しない。
noise_sigma = 0.30

def prepare_data():
    # シードを固定して再現可能にする。
    random = ROOT.TRandom3(12345)
    x_values = array("d")
    y_values = array("d")
    for i in range(n_points):
        x = x_min + (x_max - x_min) * i / (n_points - 1)
        y_true = (
            true_p0
            + true_p1 * x
            + true_p2 * x * x
        )
        y = y_true + random.Gaus(0.0, noise_sigma)
        x_values.append(x)
        y_values.append(y)
    return x,y
```

Python

フィットしてみよう

右の図が保存されるはず

右の図が出たら一旦進みます

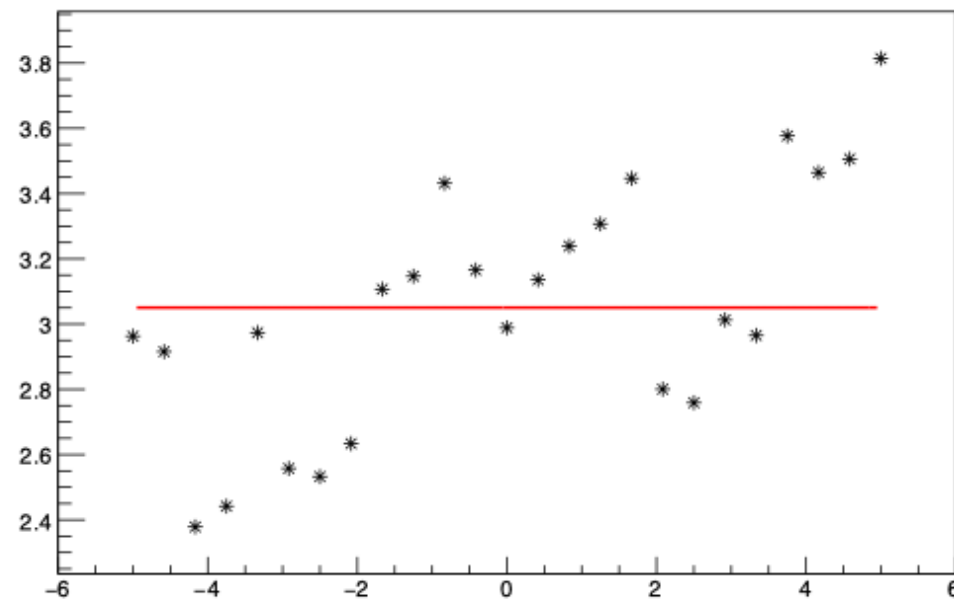
```
TF1* fitConstant = new TF1("fitConstant", "pol0", xMin, xMax);  
g->Fit(fitConstant, "R");
```

C++

```
fit_constant = ROOT.TF1("fit_constant", "pol0", x_min, x_max)  
graph.Fit(fit_constant,"R")
```

Python

Graph



パート 3：フィッティングの数学とモデル選択

どのようにモデルを選ぶべきか